

Fase 1 Diseño y arquitectura

Sistema de monitoreo y control distribuido

Curso: Internet Arquitectura y Protocolos Telemática 2026 2

Integrantes: David Rodríguez y Juan Manuel Ramírez

Fecha de entrega: 9 de septiembre de 2026

Universidad EAFIT

Introducción

El presente informe define la propuesta inicial de diseño y arquitectura para un sistema distribuido de monitoreo y control. El sistema permite que dos o más nodos reporten su estado y sus eventos a un servidor central, mientras uno o varios clientes de administración consultan la información sin comunicarse directamente con los nodos. La propuesta responde a los requisitos de la Fase 1 establecidos en el enunciado de la Entrega 1 [1].

La principal decisión de diseño consiste en combinar TCP y UDP según la naturaleza de cada intercambio. TCP se emplea en operaciones de control y consulta que requieren entrega completa y ordenada. UDP se utiliza para la telemetría periódica y para eventos críticos; en este último caso, el protocolo de aplicación agrega confirmación, temporizador, retransmisión y detección de duplicados.

Objetivos

Objetivo general

Diseñar la arquitectura y la especificación preliminar de un protocolo de capa de aplicación para supervisar nodos distribuidos mediante sockets Berkeley, resolución de nombres, autenticación, concurrencia y manejo controlado de errores.

Objetivos específicos

- Definir las entidades del sistema y sus responsabilidades.
- Establecer los tipos, el formato preliminar y las reglas de intercambio de mensajes.
- Representar el comportamiento de nodos, servidor y clientes mediante máquinas de estado legibles.
- Justificar técnicamente el uso de TCP y UDP según frecuencia, criticidad, confiabilidad y tolerancia a pérdidas.
- Prever concurrencia, autenticación, resolución de nombres, registro de actividad y control de excepciones.

1 Descripción general del problema

Una organización necesita supervisar a distancia una infraestructura formada por sensores, equipos de red, dispositivos IoT u otros componentes. Cada nodo debe enviar periódicamente métricas como utilización de CPU, temperatura, nivel de batería, disponibilidad y estado operativo. También debe informar eventos relevantes, entre ellos fallas, alarmas, pérdida de conectividad o superación de umbrales.

El servidor central registra los nodos, recibe los reportes, mantiene el estado más reciente y conserva al menos cinco registros históricos por nodo. Los clientes de administración se autentican y consultan esa información desde el servidor. No existe comunicación directa entre nodos y clientes.

La telemetría periódica tolera pérdidas ocasionales porque un reporte posterior actualiza el estado. Los eventos críticos, el registro, la autenticación y las consultas necesitan entrega confiable o mecanismos explícitos de recuperación. Esta diferencia determina la selección del transporte.

2 Arquitectura propuesta

La arquitectura se organiza en nodos de monitoreo, servidor central, clientes de administración, servicio de identidad y resolución DNS. El servidor se implementará en C y utilizará exclusivamente la API de sockets Berkeley para construir las comunicaciones de red.

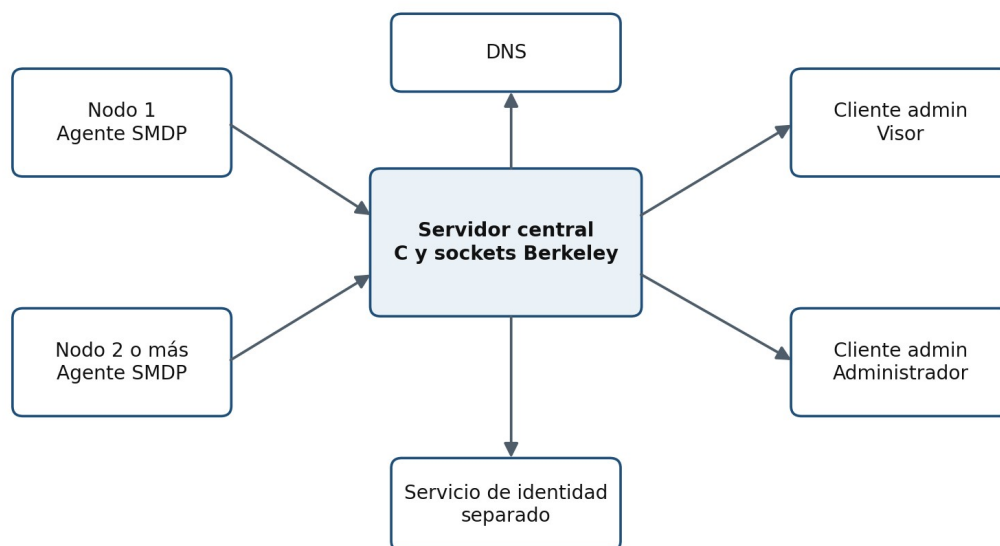


Figura 1. Arquitectura general del sistema distribuido.

La Figura 1 resume las relaciones entre entidades sin colocar rótulos sobre las flechas. Los nodos usan TCP para REG y UDP para STATUS y EVENT; los clientes usan TCP para AUTH, QUERY y respuestas. El servidor consulta el servicio de identidad y todos los componentes resuelven nombres mediante DNS.

2.1 Componentes y responsabilidades

Tabla 1. Entidades participantes.

Entidad	Responsabilidad principal	Interfaz
Nodo	Se registra, guarda su token, envía telemetría y retransmite eventos no confirmados.	TCP: REG. UDP: STATUS y EVENT.
Servidor central	Valida mensajes, conserva estado, historial y eventos, atiende consultas y genera logs.	Sockets TCP y UDP.
Cliente de administración	Se autentica, consulta información y presenta estado e historial.	TCP: AUTH, QUERY y respuestas.
Servicio de identidad	Valida credenciales y devuelve el perfil del usuario. Está separado de la aplicación principal.	Servicio accesible por nombre DNS.
DNS	Resuelve los nombres del servidor central y del servicio de identidad.	Resolución con getaddrinfo().

2.2 Concurrencia y almacenamiento

El servidor expone un socket TCP y un socket UDP. El socket TCP acepta conexiones y asigna un hilo a cada conexión para procesar REG, AUTH y QUERY. UDP no establece conexiones: un hilo receptor obtiene datagramas y los entrega a trabajadores. El estado compartido se protege con mutex o bloqueos de lectura y escritura.

La información mínima se organiza en nodos registrados, estados, eventos y sesiones. Cada nodo conserva su último estado y, como mínimo, cinco muestras históricas. El servidor se inicia con el puerto y el archivo de logs exigidos por el enunciado; TCP y UDP pueden utilizar el mismo número de puerto porque pertenecen a transportes diferentes.

2.3 Resolución de nombres

No se codifican direcciones IP. Nodos, clientes y servidor reciben nombres de dominio y los resuelven en tiempo de ejecución mediante getaddrinfo(). Si la resolución falla, el proceso registra el error, aplica un intervalo de espera y vuelve a intentarlo sin finalizar su ejecución.

2.4 Autenticación y perfiles

La autenticación se delega a un servicio de identidad separado, de modo que las credenciales no se almacenen únicamente en archivos o bases de datos internas del servidor principal. Se definen los perfiles VISOR, que consulta estado, historial y eventos, y ADMIN, que además puede consultar el registro de nodos y la información de diagnóstico.

3 Tipos de mensajes

Tabla 2. Mensajes enviados por nodos y clientes.

Mensaje	Emisor	Transporte	Propósito
REG	Nodo	TCP	Registrar identidad y capacidades.
STATUS	Nodo	UDP	Actualizar la telemetría periódica.
EVENT	Nodo	UDP	Reportar un evento crítico.
AUTH	Cliente	TCP	Validar credenciales y solicitar una sesión.
QUERY	Cliente	TCP	Consultar nodos, estado, historial o eventos.

Tabla 3. Respuestas emitidas por el servidor.

Mensaje	Transporte	Propósito
REG_OK	TCP	Confirmar el registro y devolver un token de nodo.
AUTH_OK	TCP	Confirmar autenticación, perfil y token de sesión.
AUTH_ERR	TCP	Rechazar la autenticación.
QUERY_RESP	TCP	Entregar el resultado de una consulta.
ACK	UDP	Confirmar un EVENT por su número de secuencia.
ERROR	TCP o UDP	Informar un error procesable.

4 Sintaxis preliminar del protocolo

El protocolo se denomina SMDP 1.0. Los mensajes usan UTF-8, terminan en LF y no superan 4096 bytes. En TCP, el receptor acumula bytes hasta encontrar LF y conserva los bytes posteriores para el mensaje siguiente. En UDP, cada datagrama contiene exactamente un mensaje completo.

```
SMDP/1.0|TIPO|ORIGEN|SEQ|TIMESTAMP|TOKEN|PAYLOAD_JSON\n
```

Tabla 4. Campos del mensaje SMDP 1.0.

Campo	Definición
Versión	Literal SMDP/1.0.
TIPO	Operación o respuesta definida por el protocolo.
ORIGEN	Identificador del nodo o usuario; guion si aún no existe sesión.
SEQ	Entero creciente por emisor; correlaciona solicitud y respuesta.
TIMESTAMP	Unix epoch UTC en segundos.
TOKEN	Token opaco recibido en REG_OK o AUTH_OK; guion en REG y AUTH.
PAYLOAD_JSON	Objeto JSON compacto con los datos específicos del mensaje.

4.1 Ejemplos de mensajes

```

SMDP/1.0|REG|nodo-03|1|1788912000|-|{"device_type":"sensor","location":"sala1"}\n
SMDP/1.0|STATUS|nodo-03|14|1788912045|nt_8f31|
{"cpu":32,"temp":41.5,"battery":87,"state":"ok"}\n
SMDP/1.0|EVENT|nodo-03|15|1788912050|nt_8f31|
{"event":"threshold","value":95,"threshold":90}\n
SMDP/1.0|QUERY|juan|7|1788912100|us_a104|
{"node":"nodo-03","resource":"history","limit":5}\n

```

4.2 Códigos de error

Tabla 5. Códigos preliminares de error.

Código	Significado
MALFORMED_MESSAGE	Cantidad de campos, JSON o codificación inválidos.
UNSUPPORTED_VERSION	Versión de protocolo no soportada.
UNKNOWN_TYPE	Tipo de mensaje desconocido.
NODE_NOT_REGISTERED	Nodo o token de nodo no válido.
UNAUTHORIZED	Sesión inexistente, vencida o sin permiso.
NOT_FOUND	Nodo o recurso solicitado inexistente.
INTERNAL_ERROR	Fallo interno controlado; el servidor continúa.

5 Reglas básicas de comunicación

1. El nodo resuelve el nombre del servidor. Si DNS falla, registra el error y reintenta con esperas de 1, 2, 4, 8 y máximo 30 segundos.
2. El nodo abre TCP, envía REG y espera REG_OK. El servidor devuelve un token que relaciona los datagramas UDP posteriores con el registro. La IP y el puerto se conservan para auditoría, pero no sustituyen el token.
3. STATUS se envía periódicamente por UDP. El servidor valida versión, formato, token y secuencia, actualiza el estado y conserva al menos cinco muestras. No responde con ACK salvo que exista un error.
4. EVENT se envía por UDP y requiere ACK con la misma secuencia. Si el nodo no lo recibe en un segundo, retransmite hasta tres veces. El servidor confirma los duplicados, pero almacena y procesa el evento una sola vez.
5. El cliente abre TCP, envía AUTH y espera AUTH_OK con token de sesión y perfil. Las credenciales no se escriben en los logs. La autenticación controla el acceso, pero no reemplaza el cifrado en un despliegue fuera del laboratorio.
6. Cada QUERY incluye un token de sesión válido. El servidor responde QUERY_RESP o ERROR con la misma secuencia. Una consulta histórica entrega por lo menos cinco muestras cuando existen.
7. Una lectura TCP parcial no se procesa como mensaje completo. Mensajes de más de 4096 bytes, campos inválidos o JSON incorrecto producen ERROR sin finalizar el servidor.
8. Cuando una conexión TCP cae, el hilo libera sus recursos. El cliente o nodo resuelve de nuevo el dominio y se reconecta. Un fallo individual no afecta a los demás participantes.
9. El servidor registra cada solicitud y respuesta con fecha, transporte, IP y puerto de origen, tipo, secuencia y resultado. Las contraseñas y los tokens se redactan.

6 Máquinas de estado

Las figuras siguientes separan los flujos para conservar un tamaño de lectura adecuado. Las flechas solo muestran la dirección de avance; las transiciones se explican después de cada figura.

6.1 Nodo y canal de control TCP



Figura 2. Registro del nodo mediante el canal de control TCP.

1. INICIO a RESUELTO: el nodo resuelve el dominio.
2. RESUELTO a CONECTADO: abre la conexión TCP.
3. CONECTADO a REGISTRADO: envía REG y recibe REG_OK.
4. Si DNS o TCP falla, entra en reintento con retroceso. Un ERROR detiene únicamente el registro afectado.

6.2 Nodo y envío UDP



Figura 3. Envío de telemetría y eventos del nodo mediante UDP.

1. Desde ACTIVO, el temporizador genera STATUS y el nodo vuelve a ACTIVO sin esperar ACK.
2. Cuando surge un evento, envía EVENT y pasa a ESPERA ACK.
3. Un ACK válido devuelve el nodo a ACTIVO. Un timeout causa retransmisión hasta tres veces.
4. Un ACK duplicado se ignora y un evento no confirmado se conserva para un intento posterior.

6.3 Servidor

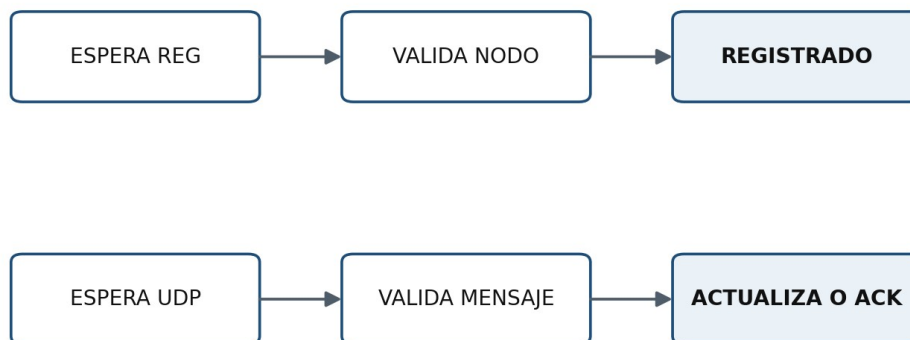


Figura 4. Flujos independientes del servidor para TCP y UDP.

La fila superior corresponde al registro TCP: el servidor espera REG, valida el nodo y responde REG_OK o ERROR. La fila inferior corresponde al receptor UDP: espera un datagrama, valida su versión, formato, token y secuencia, y luego actualiza STATUS o almacena EVENT y devuelve ACK. Un mensaje inválido no detiene ninguno de los sockets de escucha.

6.4 Cliente de administración



Figura 5. Autenticación y consulta del cliente de administración.

1. INICIO a CONECTADO: el cliente resuelve el dominio y abre TCP.
2. CONECTADO a AUTENTICADO: envía AUTH y recibe AUTH_OK.
3. AUTENTICADO a ESPERA RESPUESTA: envía QUERY.
4. QUERY_RESP o ERROR devuelve el cliente a AUTENTICADO. Una caída de TCP lleva a reconexión.

7 Análisis preliminar de TCP y UDP

Tabla 6. Selección del protocolo de transporte.

Operación	Características	Necesidad	Decisión
REG	Una por inicio; mensaje pequeño.	Entrega y respuesta confiables.	TCP. Crea el token del nodo sin implementar confiabilidad propia.
STATUS	Frecuente; mensaje pequeño.	Se tolera pérdida ocasional; interesa el dato reciente.	UDP sin ACK. Una muestra posterior reemplaza la pérdida.
EVENT	Ocasional; mensaje pequeño.	No debe perderse y puede duplicarse al retransmitir.	UDP con ACK, secuencia, timeout, tres reintentos y deduplicación.
AUTH	Una por sesión; mensaje pequeño.	Confiabilidad, orden y mantenimiento de sesión.	TCP. Una autenticación incompleta no habilita consultas.
QUERY y respuesta	Intermitente; respuesta de tamaño variable.	Entrega completa y ordenada.	TCP. Facilita transmitir historiales sin reensamblado propio.

La combinación utiliza UDP donde la información pierde valor rápidamente y TCP donde una operación incompleta impide continuar. El costo consiste en mantener dos sockets y relacionar los datagramas con el registro mediante un token. La confiabilidad de EVENT es deliberadamente limitada a las necesidades del protocolo y no pretende volver a implementar TCP.

8 Manejo de excepciones

Tabla 7. Tratamiento previsto de fallos.

Situación	Comportamiento esperado
Fallo de DNS	Registrar, esperar y reintentar sin terminar el proceso.
Conexión TCP rechazada o caída	Cerrar el descriptor, liberar el hilo y permitir reconexión.
Lectura TCP parcial	Acumular hasta LF y no procesar fragmentos.
Mensaje inválido o demasiado grande	Responder ERROR si es posible, registrar y continuar.
Datagrama duplicado o fuera de orden	Deduplicar EVENT y aceptar el STATUS más reciente.
Timeout de EVENT	Reenviar hasta tres veces y conservar el evento si se agotan.
Token inválido o vencido	Responder UNAUTHORIZED sin revelar información sensible.
Fallo de un hilo	Aislar el fallo y mantener activos los demás clientes.

9 Registro de actividad

El servidor muestra las solicitudes y respuestas en consola y también las guarda en el archivo de logs recibido como parámetro. Cada línea contiene fecha, transporte, dirección IP, puerto de origen, tipo de mensaje, secuencia, identidad y resultado. Para evitar exposición de información sensible, las contraseñas y los tokens se sustituyen por valores redactados.

```
2026-09-09T14:02:31Z          transport=UDP          peer=192.0.2.10:53001
type=EVENT seq=15 node=nodo-03 result=ACK
```

Un mutex protege la escritura del archivo para impedir que varios hilos mezclen líneas. Si el archivo deja de estar disponible, el servidor informa el problema en consola y continúa atendiendo cuando puede hacerlo de forma segura.

10 Criterios de verificación para la siguiente fase

- Dos nodos se registran y reportan simultáneamente.
- Un cliente autenticado consulta el estado y el historial de cada nodo.
- El historial entrega cinco muestras cuando existen.
- La pérdida simulada del primer ACK de EVENT causa una retransmisión y un único evento almacenado.
- Un mensaje inválido produce ERROR y el servidor continúa atendiendo.
- Una falla de DNS o TCP no termina el nodo ni el cliente.
- Dos clientes realizan consultas concurrentes sin corrupción de datos.
- La consola y el archivo de logs contienen la IP y el puerto de cada solicitud y respuesta.

Conclusiones

La propuesta SMDP 1.0 satisface los elementos solicitados para la Fase 1: descripción del problema, arquitectura, entidades, mensajes, sintaxis, reglas de comunicación, máquinas de estado y análisis de transporte. La separación entre TCP y UDP responde a las necesidades reales de confiabilidad y frecuencia de cada operación.

El diseño también establece cómo relacionar el registro TCP con los datagramas UDP, cómo autenticar clientes mediante un servicio separado, cómo atender participantes concurrentes y cómo manejar fallos sin finalizar el servidor. Estas decisiones proporcionan una base suficientemente concreta para iniciar la implementación incremental.

Referencias

[1] J. A. Vergara Tejada, "Sistema de monitoreo y control distribuido: Entrega 1 - sockets," Internet Arquitectura y Protocolos, Telemática, Universidad EAFIT, 2026.